

למידה עם חיזוקים · פרויקט גמר

חמישה שלבים, שלושה עשר אלגוריתמים

מה כל אחד עושה, ובמה בדיוק הוא שונה מהשכן שלו



1	תכנות דינמי	Value Iteration · Policy Iteration
2	מונטה קרלו	Every-visit · First-visit
3	הפרש זמני	Q-Learning · SARSA · Expected-SARSA
4	עמוק מבוסס ערך	DQN · Double · Dueling
5	מדרון מדיניות	REINFORCE · Actor-Critic · PPO

Rival Minds · מסמך התמצאות והכנה להגנה

המודל ידוע מראש, ולכן אף אחד לא לומד מניסיון. שניהם פשוט מחשבים, והמהיר יותר מנצח.

VALUE ITERATION

איטרציית ערך

מניח שאתה יודע מראש את $P(s' | s, a)$, ולכן לא צריך לשחק בכלל אלא רק לחשב. כל סריקה מפעילה את גיבוי בלמן האופטימלי על כל מצב, כלומר "ההערכה החדשה של המצב היא התוחלת של הפעולה הכי טובה ממנו". המדיניות לא נשמרת בכלל, היא פשוט נקראת חמדנית מ-v אחרי כל סריקה.

אצלו הגיבוי **סינכרוני**: קוראים מ-v הישן וכותבים לטבלה חדשה. זה מה שיוצר את הגל הוויזואלי, כי הערך מתפשט טבעת אחת לכל סריקה מהמטרה החוצה.

$$V(s) \leftarrow \max_a \sum P(s' | s, a) [r + \gamma V(s')]$$

arenas/r1_peach_castle/value_iteration.py

POLICY ITERATION

איטרציית מדיניות

מפריד את מה ש-VI מיזג לשני שלבים. **הערכה**: קח את המדיניות הנוכחית וחשב כמה היא שווה, בלי $\max$, פשוט לפי הפעולה שהמדיניות בוחרת. **שיפור**: עכשיו כשאתה יודע כמה היא שווה, החלף בכל מצב לפעולה החמדנית. חזור.

אצלו זו גרסה **קטומה**: $EVAL_SWEEPS = 8$, כלומר שמונה סריקות הערכה ואז שיפור, במקום להריץ הערכה עד התכנסות מלאה שדורשת מאות סריקות ב- $\gamma=0.98$. שווה לדעת: $EVAL_SWEEPS=1$ הוא בדיוק Value Iteration, ולכן השניים הם שני קצוות של אותו ציר.

$$V(s) \leftarrow \sum P(s' | s, \pi(s)) [r + \gamma V(s')] \quad \text{הערכה}$$

$$\pi(s) \leftarrow \operatorname{argmax}_a Q(s, a) \quad \text{שיפור}$$

arenas/r1_peach_castle/policy_iteration.py

שלב 2 · מונטה קרלו

אין מודל. לומדים מניסיון בלבד, אבל שום דבר לא מתעדכן עד שהאפיזודה נגמרת.

EVERY-VISIT MC

מונטה קרלו, כל ביקור

ההפך הגמור מתכנות דינמי: אין מודל, אין ידע מוקדם, רק ניסיון. תוך כדי האפיזודה `learn_step` רק מקליט ולא מעדכן כלום. כשהאפיזודה נגמרת רצים אחורה על המסלול, מחשבים את התשואה האמיתית, ומושכים כל $Q(s, a)$ לכיוונה.

"כל ביקור" פירושו שמעדכנים בכל הופעה של הזוג (s, a) באפיזודה, גם אם דרכת על אותה משבצת חמש פעמים. פרט מהקוד: `STEP_SCALE = 0.25`, כי צעד מלא מרעיד את Q יותר מדי כשהיעד הוא תשואה שלמה ולא מטרת TD של צעד אחד.

תשואה $G \leftarrow r + \gamma G$

עדכון $Q(s, a) \leftarrow Q(s, a) + \alpha [G - Q(s, a)]$

`arenas/r2_new_donk_city/monte_carlo.py`

FIRST-VISIT MC

מונטה קרלו, ביקור ראשון

זהה לחלוטין חוץ משורה אחת: מעדכנים רק בהופעה הראשונה של כל זוג (s, a) באפיזודה. הנימוק התיאורטי הוא שכל עדכון הוא אז אומדן בלתי מוטה של התשואה מהזוג הזה, בעוד שהחזרות ב-`every-visit` מתואמות זו לזו בתוך אותה אפיזודה.

בפועל שניהם מתכנסים, וזה בדיוק הניגוד הקלאסי מהספר שהשלב נועד להראות.

`arenas/r2_new_donk_city/first_visit_mc.py`

לא מחכים לסוף. אחרי צעד אחד כבר מעדכנים, על ידי ניחוש שמסתמך על הניחוש הקודם.

Q-LEARNING · OFF-POLICY

למידת Q

לא מחכה לסוף. אחרי צעד אחד הוא כבר מעדכן, והיעד שלו משתמש ב- $\max$: הוא שואל "מה הערך של הפעולה הכי טובה מהמצב הבא", בלי קשר למה שהמדיניות תעשה בפועל.

זה מה שהופך אותו למחוי-למדיניות: הוא מתנהג ϵ -חמדני אבל לומד את המדיניות החמדנית הטהורה. תוצאה מעשית: הוא ילמד את המסלול הקצר גם אם הוא צמוד לגומבה, כי הוא לא מתמחר את הסיכון שהוא עצמו יחליק לשם בגלל ϵ .

$$\text{target} = r + \gamma \cdot \max_a Q(ns, a')$$

arenas/r3_fossil_falls/qlearning.py

SARSA · ON-POLICY

סארסה

משנה בדיוק תו אחד ביעד: במקום $\max$ הוא לוקח את Q של הפעולה שבאמת תיבחר הלאה. זו על-המדיניות: הוא לומד את הערך של המדיניות שהוא מריץ, כולל טעויות החקירה שלה.

לכן ליד סכנה הוא לומד ערכים פחדניים יותר ובוחר מסלול בטוח יותר, כי בממוצע ϵ מהפעמים הוא כן יידרס. אצלנו ההבדל בין שני הקבצים האלה הוא ממש שורה אחת, וזו ההדגמה החזקה ביותר של הניגוד.

$$\text{target} = r + \gamma \cdot Q(ns, na)$$

arenas/r3_fossil_falls/sarsa.py

סארסה מצופה

יושב באמצע. במקום לדגום פעולה אחת (SARSA, רועש) או לקחת $\max$ (Q-Learning, אופטימי ומוטה), הוא לוקח את התוחלת תחת המדיניות הנוכחית.

אותו עדכון בתוחלת כמו SARSA, אבל בלי שונות הדגימה של הפעולה הבאה, ולכן מתכנס חלק יותר באותו גודל צעד.

$$E[Q(ns, \cdot)] = (1-\epsilon) \cdot \max_a Q(ns, a) + \epsilon \cdot \text{mean}_a Q(ns, a)$$

arenas/r3_fossil_falls/expected_sarsa.py

שלב 4 · עמוק מבוסס ערך

המצב רציף ואי אפשר להחזיק טבלה, אז רשת נוירונים מקרבת את פונקציית הערך.

רשת Q עמוקה

זהו Q-Learning כשהמצב רציף. במקום מילון יש רשת של 9 → 128 → 128 → 55 שמנבאת את כל תשעת ערכי ה-Q בבת אחת.

שלושת המייצבים הקלאסיים כולם שם: **מאגר שחזור** של 50,000 מעברים, שדגימה אקראית של 64 מתוכו שוברת את הקורלציה בין צעדים עוקבים; **רשת מטרה**, עותק קפוא שמסתנכרן כל 500 צעדים כדי שהיעד לא יזוז תוך כדי שרודפים אחריו; **וחיתוך גרדיאנט**. אצלנו יש גם $n_step = 3$, כלומר שלושה תגמולים מקופלים ליעד אחד, כך שהעונש על פגיעה מתפשט אחורה שלושה צעדים בבת אחת במקום אחד.

arenas/r4_ruined_kingdom/dqn.py

DOUBLE-DQN

DQN כפול

משנה רק את היעד. הבעיה ב-DQN רגיל: אותה רשת רועשת גם **בוחרת** את הפעולה הבאה וגם **מדרגת** אותה, אז כל פעולה שהרעש במקרה העריך גבוה מדי מנצחת את ה-max, וההערכות מנופחות שיטתית.

הפתרון הוא לפצל את התפקידים: הרשת **המקוונת** בוחרת, רשת **המטרה** מדרגת. אצלנו `q_pred_ema` עוקב בדיוק אחרי הגודל הזה, אז אפשר להראות את פער ההערכה בין השניים בגרף חי.

```
target = r + γ · Q_target( ns , argmax_a, Q_online(ns,a') )
```

`arenas/r4_ruined_kingdom/double_dqn.py`

DUELING-DQN

DQN מפוצל

משנה רק את **המבנה**, לא את חוק העדכון. הגזע מתפצל לשני ראשים: $V(s)$ שאומר כמה טוב המצב בכלל, ו- $A(s, a)$ שאומר כמה כל פעולה עדיפה.

הרעיון: בסופת טילים רוב הערך הוא "איפה אתה עומד" ולא "לאיזה צד תזוז", אז עדיף לתת לרשת ללמוד את זה ישירות. חיסור הממוצע הכרחי, כי בלעדיו הפיצול ל- V ו- A מוגדר רק עד כדי קבוע והאימון נהיה לא יציב.

```
Q(s,a) = V(s) + A(s,a) - mean_a A(s,a)
```

`arenas/r4_ruined_kingdom/dueling_dqn.py`

מפסיקים ללמוד ערכים ולומדים את ההתנהגות עצמה. דגימה מההתפלגות היא הפעולה, וגם החקירה.

REINFORCE

מדרון מדיניות מונטה קרלו

זונח את ϵ לגמרי. הרשת פולטת התפלגות על הפעולות, ודגימה ממנה היא הפעולה. זו גם החקירה, אין ϵ בכלל. אחרי אפיזודה שלמה מעלים את הלוג-הסתברות של כל פעולה ביחס לתשואה שבאה אחריה.

אין מבקר ואין bootstrapping, אז הגרדיאנט **בלתי מוטה אבל בעל שונות עצומה**, ואפיזודה אחת ברת מזל מטלטלת את כל העדכון. הריכוך היחיד אצלנו הוא הלבנת התשואות, חיסור ממוצע וחלוקה בסטיית תקן, ובנוסף אנטרופיה נגד קריסה מוקדמת למדיניות דטרמיניסטית.

$$\text{loss} = -\text{mean}_t [\log \pi(a_t | s_t) \cdot G_t] - \beta H$$

arenas/r5_tostarena/reinforce.py

ACTOR-CRITIC

שחקן ומבקר

מתקן בדיוק את השונות הזו. **שחקן** בוחר פעולות ומבקר מעריך $v(s)$, כך שכל פעולה נשפוט מול הציפייה ולא מול התשואה הגולמית. כלומר "האם זה יצא טוב יותר ממה שציפיתי מהמצב הזה".

קריטי: המבקר מבצע bootstrapping, משלים את הזנב מעבר לחלון עם v , ולכן העדכון קורה כל horizon = 64 צעדים ולא כל אפיזודה. זו ההערה החשובה מהקוד: REINFORCE-עם-בסיס הוא עדיין מונטה קרלו ואינו Actor-Critic אמיתי. מה שהופך אותו לכזה הוא ה bootstrapping (סאטון וברטו, 13.5).

$$\text{מטרה } R_t = r_t + \gamma r_{t+1} + \dots + \gamma^k V(s_{t+k})$$

$$\text{יתרון } A_t = R_t - V(s_t)$$

arenas/r5_tostarena/actor_critic.py

אופטימיזציית מדיניות פרוקסימלית

מוסיף את מה ששני הקודמים חסרים: **שימוש חוזר בטוח בנתונים**. שניהם זורקים כל דגימה אחרי צעד גרדיאנט אחד, וזה בזבזני. PPO אוסף rollout של 512 צעדים ורץ עליו **ארבעה מחזורים** במיני-אצוות של 128.

אבל שימוש חוזר מסוכן, כי אחרי העדכון הראשון המדיניות כבר לא זו שאספה את הנתונים. הפתרון הוא **חיתוך יחס ההסתברויות** לטווח $[0.8, 1.2]$: ברגע שהמדיניות החדשה התרחקה מספיק מהישנה, הגרדיאנט של אותה דגימה **נכבה**, כך שאף אצווה לא יכולה לדחוף את המדיניות מעבר לצוק. בנוסף GAE עם $\gamma = 0.95$ לאיזון הטיה מול שונות באומדן היתרון.

$$\text{loss} = -\min(\text{ratio} \cdot A, \text{clamp}(\text{ratio}, 1-\epsilon, 1+\epsilon) \cdot A)$$

[arenas/r5_tostarena/ppo.py](#)

החוט המקשר, במשפט לכל שלב

1. יודע את המודל, ולכן **מחשב** במקום לנסות.
2. לא יודע ולכן **מנסה**, אבל לומד רק כשהאפיזודה נגמרת.
3. לומד **בכל צעד**, על ידי ניחוש שמסתמך על הניחוש הקודם.
4. מחליף את הטבלה **ברשת** כשהמצב רציף ואי אפשר לספור מצבים.
5. מפסיק ללמוד ערכים ולומד את **ההתנהגות** עצמה.